

Actividad de Clase: Subconsultas en SQL para Auditoría de Datos

Una empresa de retail necesita auditar la calidad de sus datos en la base de clientes y pedidos. Para ello, debes aplicar subconsultas en SQL que permitan obtener métricas y depurar información: detectar clientes sin pedidos, calcular promedios de precios, identificar ventas superiores al percentil, y realizar actualizaciones y eliminaciones de registros con base en condiciones dinámicas.

Objetivos y Preparación

Objetivos de Aprendizaje

- Reconocer cuándo es recomendable utilizar subconsultas en lugar de joins
- Construir consultas SQL anidadas para obtener y modificar datos (SELECT, INSERT, UPDATE, DELETE)
- Diagnosticar errores comunes y validar la consistencia de los resultados

Materiales Necesarios

- Cuenta en PostgreSQL/pgAdmin o entorno equivalente (Colab con conexión a BD opcional)
- Tablas de ejemplo: clientes, pedidos, empleados, productos, ventas
- Lectura previa de la clase 4: Subconsultas en SQL

Requisitos Técnicos

- Motor PostgreSQL 13+ o servicio equivalente
- Scripts de creación de tablas y datos básicos (si no los tienes, indícalo y te los proporciono)

Tiempo total: 45 minutos

Formato de entrega: Archivo SQL con las consultas + documento breve (Word/PDF) con reflexión.

Instrucciones Paso a Paso

01

Consulta correlacionada

Recupera productos cuyo precio es mayor al promedio de su categoría.

Micro-check: ¿La subquery devuelve un valor por cada categoría correctamente?

02

INSERT con subquery escalar

Inserta en una tabla de auditoría los empleados cuyo sueldo supere el percentil 90 de la empresa.

Micro-check: ¿Validaste que el percentil se calcule sobre todos los registros actuales?

03

UPDATE con subquery tabular

Reduce en 10% el stock de productos con ventas totales superiores al promedio general de ventas.

Micro-check: ¿La subquery en FROM tiene alias asignado?

04

DELETE con subquery NOT EXISTS

Elimina pedidos cancelados de clientes inexistentes en la tabla de clientes.

Micro-check: ¿Tu condición de borrado asegura que no elimine pedidos válidos?

05

Reflexión final

Documenta en un párrafo cuándo elegirías subqueries frente a joins en proyectos reales de datos.

Criterios de Éxito y Errores Comunes

Criterios de Éxito

- Las consultas corren sin errores y entregan el resultado esperado
- Uso correcto de alias en subqueries derivadas
- Aplicación de operadores EXISTS, IN, ANY o ALL según corresponda
- Justificación clara en la reflexión escrita

Errores Comunes a Evitar

- No asignar alias a tablas derivadas
- Subqueries escalares que devuelven más de una fila
- Usar JOIN cuando la lógica de negocio exige un filtro semántico con EXISTS/NOT EXISTS
- No validar índices y cardinalidades

Rúbrica de Evaluación

Criterio	Excelente (25%)	Competente (20%)	Básico (10%)	Ponderación
1. Comprensión del concepto clave	Explica con claridad el rol de cada subquery y justifica su elección frente a joins.	Identifica correctamente el uso de subqueries pero sin mayor justificación.	Reconoce la subquery, pero la confunde con un JOIN.	25%
2. Aplicación técnica	Todas las consultas se ejecutan correctamente y son reproducibles.	La mayoría se ejecuta correctamente con pocos ajustes.	Varias consultas presentan errores de sintaxis o lógica.	35%
3. Calidad del entregable	Entrega ordenada, con código comentado y reflexión final clara.	Entrega con código completo pero poco documentado.	Entrega incompleta o con escasa organización.	20%
4. Justificación/interpretación	Reflexión final bien argumentada con ejemplos de uso en la industria.	Reflexión general pero sin ejemplos claros.	Reflexión superficial, limitada a repetir teoría.	20%

Total: 100%